📚 MANUAL TÉCNICO DEL PROGRAMADOR Y ARQUITECTURA DE CÓDIGO

Proyecto: Mini Juego "Piedra, Papel o Tijera" con Persistencia en Disco (`juegopiedra`)

Institución: E.E.S.T. N°1 — Vicente López

Asignatura: Laboratorio de Programación (LPR) — 5° Año

Docente: Mansilla Muñoz York Elías

Autor / **Grupo:** Alexander Veit / GAMY

Fecha: 23 de octubre de 2026 | **Versión:** v1.0.0

🏗️ 1. ARQUITECTURA DEL SISTEMA Y ENTORNO DE DESARROLLO

Requisitos del Entorno

- **Compilador:** `g++` (MinGW-w64 GCC v8.1.0 o superior con soporte C++11).
- **Entorno Integrado:** Visual Studio Code o terminal PowerShell 5.1+.
- **Librerías Estándar C++:** `<iostream>`, `<fstream>`, `<string>`, `<cstdlib>`, `<ctime>`.

Estructura del Repositorio (`juegopiedrapapelotijera`)

```
piedrapapelotijera/
├── .gitattributes
├── .gitignore
├── LICENSE
├── README.md
├── docs/
│ ├── InformeEEST1_LPR2026_ACT07_G03_Informe_v1.0.0.pdf
│ ├── CHANGELOG.md
│ └── manuales/
│ ├── manual_programador_v1.0.0.pdf
│ ├── manual_usuario_v1.0.0.pdf
│ ├── manual_programador_v1.0.0.md
│ └── manual_usuario_v1.0.0.md
├── src/
│ ├── main.cpp
│ └── historial_partidas.txt
└── capturas/
├── jugada_consola.png
└── archivo_disco.png
```

---

2. ESPECIFICACIÓN DE TIPOS Y MÓDULOS (`src/main.cpp`)

2.1 Estructura de Datos `Partida`

Agrupa los atributos heterogéneos de cada evento de juego:

```
struct Partida {
    int idPartida;
    string jugador;
```

```
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

- Alineación en memoria: Los miembros residen contiguos en memoria volátil (RAM) durante la ejecución de la partida.
- Complejidad espacial: O(1) por cada objeto instanciado en el ámbito de la función.

2.2 Módulo void jugarPartida(int& contadorID)

Parámetro: Pass-by-reference (int& contadorID) para autoincrementar secuencialmente el identificador global de partidas 1, 2, 3, …, N. Manejo de Búfer: Invoca cin.ignore() para limpiar el carácter de salto de línea pendiente (\n) acumulado por el cin >> opcion previo. Generación de IA: Genera un valor pseudoaleatorio en el rango [1, 3]:

```
opcIA = (rand() % 3) + 1;
```

Evaluación de Reglas: Mapea enteros a cadenas ("Piedra", "Papel", "Tijera") y determina la victoria, empate o derrota mediante comparaciones condicionales.

2.3 Módulo void guardarEnDisco(const Partida& p)

Flujo de Salida: ofstream archivo("historial_partidas.txt", ios::app);

Modo de Apertura: ios::app (Append) ubica el puntero de escritura al final de la secuencia de bytes del archivo en disco. Formato de Salida: CSV separado por comas: idPartida,jugador,eleccionJugador,eleccionIA,resultado\n

Tratamiento de Errores: Valida !archivo.is_open() imprimiendo un mensaje de fallo en el flujo de error estándar cerr.

2.4 Módulo void leerHistorialDisco()

Flujo de Entrada: ifstream archivo("historial_partidas.txt");

Lectura Secuencial: Utiliza la sobrecarga de getline(stream, variable, delimitador) especificando ',' como delimitador de campo. Complejidad Temporal: O(N) donde N es la cantidad total de registros grabados en disco.

Compilación con estándar C++11

g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe

Ejecución directa del binario

.\src\piedrapapelotijera.exe

Referencias

Stroustrup, B. (2013). The C++ programming language (4th ed.). Addison-Wesley. Microsoft Corporation. (2026). basic_ofstream Class. https://learn.microsoft.com/cpp/standard-library/basic-ofstream-class